

Yesss 🍆🔥 Let's do the **10-scenario cache quiz**.

Don't overthink it. For each one, answer these **4 things**:

1. Static or Dynamic?
2. Data Cache? ☒ / ☒
3. Full Route Cache? ☒ / ☒
4. Router Cache? ☒ / ☒



Tiny reminder before you start

- `revalidate` → data can be cached
- `no-store` → that fetch isn't cached
- `cookies()` / `headers()` → request-dependent rendering
- Static route → can participate in Full Route Cache
- Router Cache → **client-side RSC payload cache**, so think about whether the scenario involves a client navigation/loaded route



Scenario 1 — Completely static

```
// app/about/page.tsx

export default function About() {
  return <h1>About Us</h1>;
}
```

No fetches.

No cookies.

No headers.

No search params.

Your answer:

1. Static/Dynamic:
2. Data Cache:
3. Full Route Cache:
4. Router Cache:



Scenario 2 — Cached API data

```
export default async function Products() {  
  const res = await fetch("https://api.example.com/products", {  
    next: { revalidate: 60 }  
  });  
  
  const products = await res.json();  
  
  return <ProductList products={products} />;  
}
```

Your answer:

1. Static/Dynamic:
2. Data Cache:
3. Full Route Cache:
4. Router Cache:



Scenario 3 — no-store

```
export default async function Products() {  
  const res = await fetch("https://api.example.com/products", {  
    cache: "no-store"  
  });  
  
  const products = await res.json();  
  
  return <ProductList products={products} />;  
}
```

Your answer:

1. Static/Dynamic:
2. Data Cache:
3. Full Route Cache:
4. Router Cache:



Scenario 4 — Cookie

```
import { cookies } from "next/headers";

export default async function Dashboard() {
  const cookieStore = await cookies();

  const session = cookieStore.get("session");

  return <h1>Hello {session?.value}</h1>;
}
```

Your answer:

1. Static/Dynamic:
2. Data Cache:
3. Full Route Cache:
4. Router Cache:



Scenario 5 — Dynamic route ≠ dynamic rendering

app/products/[id]/page.tsx



```
export default async function ProductPage() {
  return <h1>Product</h1>;
}
```

No:

cookies()
headers()
searchParams
no-store



No dynamic APIs.

Your answer:

1. Static/Dynamic:
2. Data Cache:
3. Full Route Cache:
4. Router Cache:



💡 Remember: `[id]` tells you about **route structure**, not automatically rendering.

✎ Scenario 6 — Dynamic route + cached data

app/products/[id]/page.tsx



```
export default async function ProductPage({
  params
}: {
  params: Promise<{ id: string }>
}) {
  const { id } = await params;

  const res = await fetch(
    `https://api.example.com/products/${id}`,
    {
      next: { revalidate: 300 }
    }
  );

  const product = await res.json();

  return <h1>{product.name}</h1>;
}
```

Assume we're **not using** `generateStaticParams()`.

Your answer:

1. Static/Dynamic:
2. Data Cache:
3. Full Route Cache:
4. Router Cache:



✎ Scenario 7 — Cookie + public products

```
import { cookies } from "next/headers";

export default async function Products() {
  const cookieStore = await cookies();
}
```

```
const theme = cookieStore.get("theme");

const res = await fetch(
  "https://api.example.com/products",
  {
    next: { revalidate: 60 }
  }
);

const products = await res.json();

return (
  <div className={theme?.value}>
    <ProductList products={products} />
  </div>
);
}
```

Think carefully here. 🐱

Your answer:

1. Static/Dynamic:
2. Data Cache:
3. Full Route Cache:
4. Router Cache:



Scenario 8 — Two fetches, different caching

```
export default async function ProductPage() {
  const product = await fetch(
    "https://api.example.com/product/1",
    {
      next: { revalidate: 300 }
    }
  );

  const recommendations = await fetch(
    "https://api.example.com/recommendations",
    {
      cache: "no-store"
    }
  );
}
```

```
return <ProductPageUI />;  
}
```

Your answer:

1. Static/Dynamic:
2. Data Cache:
3. Full Route Cache:
4. Router Cache:



Scenario 9 — Search params

```
export default async function SearchPage({  
  searchParams  
}: {  
  searchParams: Promise<{ q?: string }>  
}) {  
  const { q } = await searchParams;  
  
  return <h1>Results for {q}</h1>;  
}
```

For example:

```
/search?q=shoes
```



Your answer:

1. Static/Dynamic:
2. Data Cache:
3. Full Route Cache:
4. Router Cache:



Scenario 10 — The tricky one

```
import { cookies } from "next/headers";  
  
export default async function ProductPage() {  
  const cookieStore = await cookies();  
}
```

```
const session = cookieStore.get("session");

const product = await fetch(
  "https://api.example.com/product/1",
  {
    next: { revalidate: 300 }
  }
);

const cart = await fetch(
  "https://api.example.com/cart",
  {
    cache: "no-store"
  }
);

return <ProductPageUI />;
}
```

There are **three different concepts** mixed together:

cookies()
cached product fetch
uncached cart fetch



🔥 Give me your prediction.

1. Static/Dynamic:
2. Data Cache:
3. Full Route Cache:
4. Router Cache:



🎯 **Answer all 10 in one shot**

You can literally reply:

- 1) Static, ❌, ✅, ✅
- 2) ...
- 3) ...
- ...
- 10) ...



Then I'll **grade each one** and **explain only the places where your mental model needs correction**. No spoon-feeding. 🦊 🧠 🔥



- 1) static,no data cache,full route cache=yes,router cache=yes
- 2)static,yes,yes,yes
- 3)dynamic,no,no,no
- 4)dynamic,no,no,no
- 5)static,yes,yes,yes
- 6)dynamic,cache,yes,yes
- 7)dynamic,yes,yes,yes
- 8)dynamic,yes for 1 no for 2,same for all 3
- 9)dynamic,no,no,no
- 10)static,1 cache 2 no chache same for all

Yesss 😎🔥 You're actually very close. The **biggest thing to fix is your understanding of Router Cache** and one important point around **dynamic routes / Full Route Cache**.

Let's grade them one by one.

🔧 Results

#	Your answer	Verdict
1	Static, no Data, Full Route ✓ , Router ✓	🟢
2	Static, Data ✓ , Full Route ✓ , Router ✓	🟢
3	Dynamic, Data ✗ , Full Route ✗ , Router ✗	🟢 *
4	Dynamic, Data ✗ , Full Route ✗ , Router ✗	🟢 *
5	Static, Data ✗ , Full Route ✓ , Router ✓	🟡
6	Dynamic, Data ✓ , Full Route ✓ , Router ✓	🔴
7	Dynamic, Data ✓ , Full Route ✓ , Router ✓	🔴
8	Dynamic, Data mixed, Full Route same	🟢 / 🟡
9	Dynamic, no Data, no Full Route, no Router	🟢 *
10	Static, Data mixed, Full Route same	🔴

But before we correct them, there's a **very important ambiguity in the quiz itself**.

First: Router Cache is NOT simply "yes for every route"

This is the biggest correction.

You were answering:

"If the route exists, Router Cache = yes."

Not quite.

Router Cache is a **client-side cache of RSC payloads** associated with the App Router's client navigation state.

So whether it contains something depends on **what the browser has already visited/received**, not simply whether the route is static.

Think:

Server-side:

Data Cache
Full Route Cache

Client-side:

Router Cache



Therefore:

Static $\neq$ automatically Router Cache
Dynamic $\neq$ automatically no Router Cache



A dynamically rendered route can still have its RSC payload in the client's Router Cache after navigation.



That's a key distinction.

1 Static page

`</>` TypeScript

```
export default function About() {  
  return <h1>About Us</h1>;  
}
```



```
}
```

You said:

Static
Data Cache ❌
Full Route Cache ✅
Router Cache ✅



Server-side:

Data Cache ❌
Full Route Cache ✅



Correct.

For Router Cache:

The **route's RSC payload** can be stored in the Router Cache once the client receives it.

So your conceptual answer is fine if we're asking:

"Can this route's RSC payload be stored in Router Cache?"

● Good.

2 revalidate: 60

```
</> TypeScript  
  
fetch(url, {  
  next: { revalidate: 60 }  
})
```



You said:

Static
Data Cache ✅
Full Route Cache ✅
Router Cache ✅



● **Correct**, assuming the route has no other dynamic behavior.

Mental picture:

```
API
↓
Data Cache ✓
↓
Render
↓
Full Route Cache ✓
↓
RSC Payload
↓
Router Cache
```



Nice. 🔥

3 no-store

You said:

```
Dynamic
Data Cache ✗
Full Route Cache ✗
Router Cache ✗
```



The first three are correct in the traditional/current conceptual model:

```
no-store
↓
Data Cache ✗
↓
Dynamic rendering
↓
Full Route Cache ✗
```



But:

```
Router Cache ✗
```



is the part to correct.

A dynamically rendered page can still have its RSC payload cached **in the client Router Cache** after the browser receives it.

So don't make:

Dynamic → Router Cache ❌



your rule.

- **Static/Dynamic:** correct
- **Data Cache:** correct
- **Full Route Cache:** correct
- **Router Cache:** depends on client navigation/state

4 cookies()

You said:

Dynamic
Data ❌
Full Route ❌
Router ❌



First two cache conclusions need nuance.

`cookies()` makes the route **request-dependent/dynamic** in the relevant caching model.

Therefore:

Full Route Cache ❌



But `cookies()` itself does **not mean Data Cache = ❌**.

Remember your previous question?

We said:

`cookies()`
↓
dynamic rendering

BUT

`fetch(...revalidate)`
↓
Data Cache can still exist



So:

```
cookies() ≠ Data Cache disabled
```



In this particular scenario, though, there is **no fetch at all**, so there simply isn't any Data Cache entry.

Therefore:

```
Data Cache → not applicable
```



Router Cache again depends on the client.

5 [id] but no generateStaticParams()

This one was:

```
/products/[id]
```



with:

```
</> TypeScript
```



```
export default async function ProductPage() {  
  return <h1>Product</h1>;  
}
```

You answered:

```
Static  
Data Cache yes  
Full Route Cache yes  
Router Cache yes
```



The important part:

[id] **does NOT** automatically mean dynamic rendering.

You got that. 🎯

But there is **no data fetch**, so:

```
Data Cache  
  ↓  
not applicable
```



not "yes."

The route can be statically rendered **for paths that are generated/handled according to the route's configuration**, but without `generateStaticParams`, you shouldn't blindly conclude that every arbitrary `/products/123` is prebuilt at build time.

This is exactly why we separated:

Dynamic route
≠
Dynamic rendering



🟡 Your **Static** intuition is good, but the Data Cache answer is wrong.

6 Dynamic route + `revalidate: 300`

This was:

`/products/[id]`



and:

```
</> TypeScript

fetch(..., {
  next: { revalidate: 300 }
})
```



You answered:

Dynamic
Data Cache ✅
Full Route Cache ✅



This is the important correction:

`[id]` **doesn't make it dynamically rendered.**

And:

`generateStaticParams()`



was explicitly absent.

But absence of `generateStaticParams()` **doesn't by itself mean "dynamic rendering."**

The `fetch` is cacheable:

Data Cache 



The route may still be statically rendered / cached depending on the route's configuration and modern Next.js behavior.

So your:

Dynamic



was too quick.

🔥 This is exactly the trap we wanted this quiz to expose.

7 Cookie + public products

You answered:

Dynamic

Data Cache 

Full Route Cache 



This is the one we literally discussed five minutes ago. 😂

`</>` TypeScript

```
cookies()
```



means:

request-dependent rendering

↓

Dynamic



And:

`</>` TypeScript

```
fetch(..., {  
  next: { revalidate: 60 }  
})
```



means the **products data can be cached**.

So:

Dynamic rendering
+
Data Cache ✓



But:

Full Route Cache ✗



because the overall rendered result depends on the cookie.

So:

Dynamic
Data Cache ✓
Full Route Cache ✗
Router Cache → client-dependent



This is probably the most important scenario in the whole quiz.

8 Cached product + uncached recommendations

You said:

Dynamic
Data:
 fetch 1 ✓
 fetch 2 ✗
Full Route: same



The data part is correct:

product
↓
Data Cache ✓

recommendations
↓
Data Cache ✗



But why did you say Dynamic?

Because of:

```
</> TypeScript
```



```
cache: "no-store"
```

That is indeed a strong signal for dynamic rendering in the classic App Router caching model.

So:

Dynamic



Data Cache:

product



recommendations



Full Route Cache:



Your reasoning here was basically right. 👍

9 Search params

You answered:

Dynamic



Data



Full Route



Router



This one needs a modern-Next.js nuance.

`searchParams` makes the route depend on request URL state in the relevant model, so you should treat it as **dynamic/request-dependent** for this mental model.

There is no fetch:

Data Cache → none



And the route's full rendered output isn't a single universally reusable Full Route Cache entry in the simple model.

But again:

Router Cache



isn't the right blanket rule.

The client can cache RSC payloads it receives.

So:

- Dynamic
- No Data Cache
- No Full Route Cache
- Router Cache depends on client state/navigation

10 The tricky one

This was:

</> TypeScript

```
cookies()
```

```
fetch(product, { revalidate: 300 })
```

```
fetch(cart, { cache: "no-store" })
```

You answered:

Static

Data:

product ✓

cart ✗

Full Route: same

The data part is good:

product

↓

Data Cache ✓

cart

↓

Data Cache ✗

But:

```
cookies()
```

and:

cart no-store



make the overall route request-dependent/dynamic in the conceptual model.

Therefore:

Dynamic

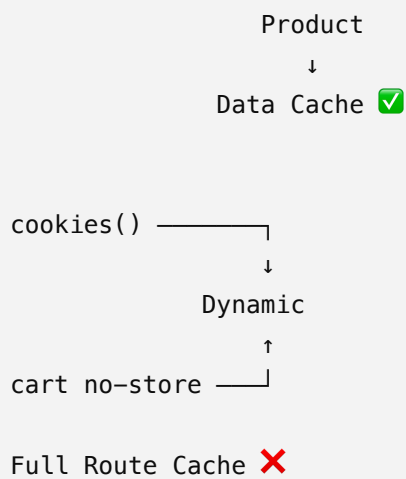


and:

Full Route Cache ❌



So:



Your BIGGEST takeaway

I see exactly where your mental model is currently getting tangled.

You're sometimes treating:

Data Cache
Full Route Cache
Router Cache



as if they're **three stages that must all say YES or NO together.**

They're not.

Instead:

Question 1 — Rendering

Does the route depend on request-specific information?



↓

Static / Dynamic

Question 2 — Data

Is this particular data fetch cacheable?



↓

Data Cache?

Question 3 — Server-rendered route

Can the rendered route output be cached?



↓

Full Route Cache?

Question 4 — Browser

Has this client received/retained the RSC payload?



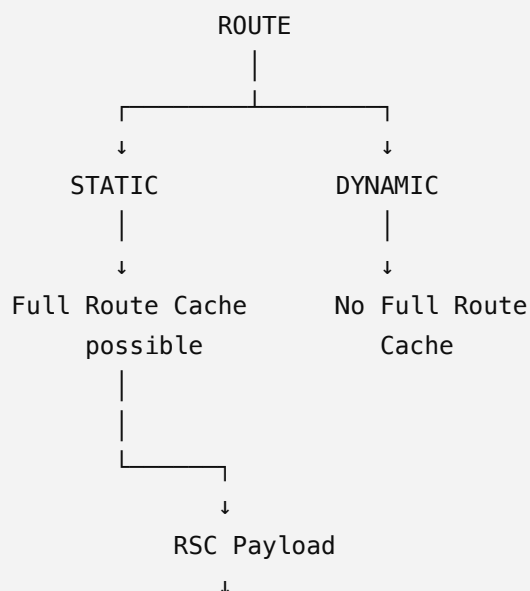
↓

Router Cache?



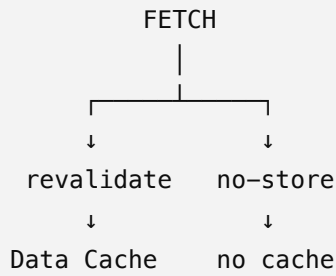
Four independent questions.

The cheat sheet



Router Cache
(client-side)

And separately:



★ The sentence I want you to remember:

Rendering tells us whether the route is static/dynamic. Data caching tells us whether individual data can be reused. Full Route Cache concerns the rendered route. Router Cache concerns the client's RSC payload.

You're getting the architecture now. The mistakes you're making are mostly **mixing the layers**, not misunderstanding the individual concepts—which is actually a good sign. 🤔



Let's do it 🤖🔥 This is the piece that makes **Router Cache** finally click.

Layer 2.5 — `<Link>` Navigation vs Refresh

We'll imagine this app:

```
/products
/products/123
/cart
```



And assume `/products/123` is dynamically rendered but its product data is cached.

🟢 Step 1 — User visits `/products`

The user initially opens:

```
https://example.com/products
```



For a simplified flow:

```
Browser
  ↓
CDN / Edge
  ↓
Next.js
  ↓
render / cached route
  ↓
HTML + RSC Payload
  ↓
Browser
```



The browser receives:

```
HTML
+
RSC Payload
```



The App Router can now keep relevant RSC information in:

Router Cache



/products → RSC Payload



So now:

SERVER

CLIENT



Data Cache



Full Route Cache



HTML + RSC



Router Cache



/products

● Step 2 — User clicks `<Link href="/products/123">`

Now they click:

`</>` TypeScript



```
<Link href="/products/123">
  iPhone
</Link>
```

This is **client-side navigation**.

The browser doesn't simply behave like a traditional:

 HTML

`</>`



```
<a href="/products/123">
```

where the browser performs a full document navigation.

Instead, Next.js can request the required **RSC payload** and update the relevant UI.

Conceptually:

Click Link



Next.js Router



request RSC payload



```
↓  
Next.js server  
↓  
RSC Payload  
↓  
Browser  
↓  
Router Cache
```

Now:

Router Cache



```
/products    → RSC  
/products/123 → RSC
```

Step 3 — User clicks `/cart`

Again:

 TypeScript



```
<Link href="/cart">  
  Cart  
</Link>
```

Client-side navigation:

```
Browser  
↓  
Next.js Router  
↓  
request needed RSC  
↓  
Server  
↓  
RSC Payload  
↓  
Router Cache
```



Now the client might have:

Router Cache



```
/products
```

```
/products/123  
/cart
```

🟢 Step 4 — User goes back to `/products/123`

Now something interesting can happen.

The client already has:

```
/products/123  
↓  
RSC Payload
```



in its Router Cache.

So the client can potentially reuse that cached client-side route data rather than fetching it again.

That's one reason client-side navigation can feel **really fast**. 🚀

🔥 Now compare that with REFRESH

This is where you need to separate the concepts.

Suppose you're at:

```
/products/123
```



and press:

```
Cmd + R
```



That's a **full document refresh**.

The browser starts a new request:

```
Browser  
↓  
CDN / Edge  
↓  
Next.js
```



The existing in-memory Router Cache isn't something you should think of as surviving that full browser reload.

So:

```
Refresh
  ↓
new document request
  ↓
server/CDN involved
```



Whereas:

```
<Link>
  ↓
client-side navigation
  ↓
RSC navigation
  ↓
Router Cache can participate
```



This gives you the key difference

`<Link>`

```
Client navigation
  ↓
RSC payload
  ↓
Router Cache
  ↓
update UI
```



Browser refresh

```
Full document request
  ↓
CDN / server
  ↓
HTML + RSC
  ↓
new client session/state
```





But don't say "Link never hits the server"

This is a common interview mistake.

Don't say:

"`<Link>` means no server request."



Not necessarily.

Instead say:

"`<Link>` enables client-side navigation. Next.js can request the required RSC payload from the server rather than performing a full document navigation."

That's much more accurate.



Now put all the caches into our example

Suppose:

`/products/123`



has:

Product data



Data Cache



and the route is dynamically rendered:

Full Route Cache



The user navigates with `<Link>` :

`<Link href="/products/123">`



Flow:

Browser

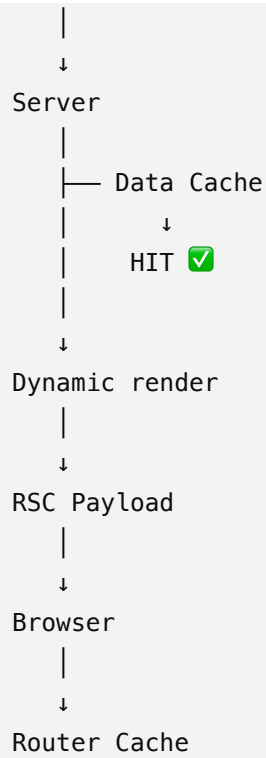


client navigation



Next.js Router





Notice something beautiful here:

Dynamic rendering
+
Data Cache HIT
+
Router Cache

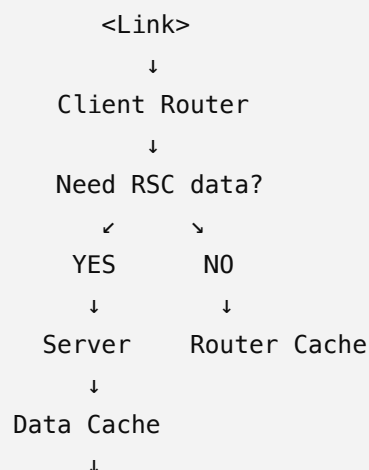


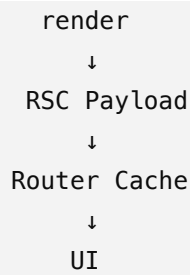
can all coexist.

🔥 This is exactly why we kept the layers separate.

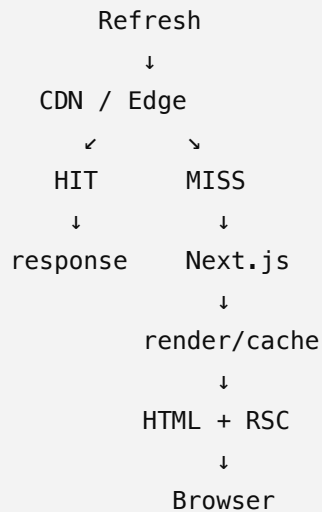
🧠 Your final mental model

When a user clicks a `<Link>`:





When the user refreshes:



★ Interview-ready sentence

If they ask:

"What's the purpose of the Router Cache?"

Say:

"The Router Cache is a client-side cache used by the Next.js App Router to retain RSC payloads for visited or prefetched route segments, which helps make client-side navigation faster and lets the router preserve navigation state."

That's a very solid answer. 😎🔥



ok

Perfect 😎🔥

At this point, you've got the **four cache layers + rendering decision** pretty well separated.

The next piece I'd teach is **cache invalidation**, because that's where everything becomes practical:

Data Cache

↓

How do I invalidate it?

Full Route Cache

↓

How does it get regenerated?

Router Cache

↓

How does the client get updated?

CDN

↓

How does HTTP caching/revalidation fit in?



Then we'll connect that to:

```
revalidatePath()  
revalidateTag()  
updateTag()  
refresh()
```



and especially **Server Action vs Route Handler**, which was one of your earlier sticking points. 🔥



but we covered this topics right

Yes! 🤪🔥 **We already covered cache invalidation.** You're right.

Earlier we went through:

```
revalidateTag()  
updateTag()
```



```
revalidatePath()
refresh()
```

and also discussed:

Server Action vs Route Handler



including how invalidation differs depending on where you call them.

So we shouldn't redo that. 👍

Where we are now

You've essentially covered:

- ✅ Layer 1
 - Route structure
 - Dynamic segments
 - Static vs dynamic rendering
 - Data caching
- ✅ Layer 2
 - Data Cache
 - Full Route Cache
 - Router Cache
 - CDN / HTTP Cache
 - Edge vs origin
 - TTL
 - Revalidation
 - <Link> navigation vs refresh
- ✅ Cache invalidation
 - revalidatePath
 - revalidateTag
 - updateTag
 - refresh
 - Server Action vs Route Handler



 The next NEW topic should be:

Modern Next.js caching — Cache Components

Specifically:

```
"use cache"
  ↓
cacheLife()
  ↓
cacheTag()
```



↓
PPR / Partial Prerendering
↓
How this differs from the older
Data Cache / Full Route Cache model

This is important because we're intentionally learning **modern Next.js**, rather than accidentally drilling an older caching model into your head. 😎

So yes — **skip invalidation repetition. We move forward to modern Cache Components.** 🔥



ChatGPT can make mistakes. Check important info.